LAB ASSIGNMENT 1

Time Complexity Analysis — C++ Programs

1. Linear Search — O(n)

Measures the execution time for array sizes 10,000, 50,000 and 100,000. The target is absent, so the program checks the entire array (worst case).

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

int main() {
    for (int n : {10000, 50000, 100000}) {
        int *a = new int[n];
        for (int i = 0; i < n; i++) a[i] = i;

        auto start = high_resolution_clock::now();
        int target = -1, pos = -1;
        for (int i = 0; i < n; i++)
            if (a[i] == target) { pos = i; break; }
        auto stop = high_resolution_clock::now();

        cout << "n=" << n << ", Time: "
             << duration_cast<microseconds>(stop-start).count()
             << " microseconds\\n";
        delete[] a;
    }
}
```

2. Binary Search — O(log n)

Uses a sorted array and measures execution time for the same data sizes. The absent target tests the search's worst-case path.

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

int main() {
    for (int n : {10000, 50000, 100000}) {
        int *a = new int[n];
        for (int i = 0; i < n; i++) a[i] = i;

        auto start = high_resolution_clock::now();
        int target = -1, l = 0, r = n-1, pos = -1;
        while (l <= r) {
            int mid = l + (r-l)/2;
            if (a[mid] == target) { pos = mid; break; }
            if (a[mid] < target) l = mid+1;
            else r = mid-1;
        }
        auto stop = high_resolution_clock::now();

        cout << "n=" << n << ", Time: "
             << duration_cast<microseconds>(stop-start).count()
             << " microseconds\\n";
        delete[] a;
    }
}
```

3. Nested Loops — O(n²)

Counts operations in two nested loops for increasing values of n to demonstrate quadratic growth. Sizes are smaller because the number of operations grows rapidly.

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace chrono;

int main() {
    for (int n : {1000, 5000, 10000}) {
        long long count = 0;
```

```
        auto start = high_resolution_clock::now();

        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                count++;

        auto stop = high_resolution_clock::now();
        cout << "n=" << n << ", Time: "
             << duration_cast<microseconds>(stop-start).count()
             << " microseconds\\n";
    }
}
```

Result / Conclusion

• Linear Search: O(n) — work grows approximately in proportion to n.
• Binary Search: O(log n) — requires far fewer comparisons on sorted data.
• Nested Loops: O(n^2) — work grows approximately with the square of n.

Note: Actual timings depend on the computer, compiler and optimization settings. Execution times are measured in microseconds; very short runs may report 0 microseconds.